



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: Pablo Arturo Gonzalez Cuellar

N° de Cuenta: 319241013

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 26/10/2025

CALIFICACIÓN: _____

1.- Agregar un spotlight (que no sea luz de color blanco ni azul) que parta del cofre de su coche y al abrir y cerrar el cofre ilumine en esa dirección.

Para esta primera parte la práctica primero declaramos el movimiento de la apertura del cofre en nuestra clase Windows.h

```
GLfloat getmuevex() { return muevex; }
GLfloat getarticulacionCofre() { return articulacionCofre; }
int getTeclaPresionada() { return tecla_presionada; }

GLfloat movx;
GLfloat articulacionCofre;
```

Para después agregar la lógica de los movimientos en Windows.cpp

```
if (key == GLFW_KEY_1)
{
    theWindow->movx -= 1.0;
}
if (key == GLFW_KEY_2)
{
    theWindow->movx += 1.0;
}

if (key == GLFW_KEY_F)
{
    static int signo = -1; // bandera
    if (theWindow->articulacionCofre <= -60.0f)
        signo = 1;
    else if (theWindow->articulacionCofre == 0.0f)
        signo = -1;

    theWindow->articulacionCofre += (signo * 5.0f);
}
```

Y así mandar a llamar en nuestro main al movimiento del cofre

```
model = modelaux;
model = glm::translate(model, glm::vec3(-2.8f, 2.9f, 0.0f));
model = glm::scale(model, glm::vec3(1.8f, 1.8f, 1.8f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacionCofre()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
cofreTexture.UseTexture();
Cofre_Bob.RenderModel();
```

Una vez creado el movimiento de nuestro cofre, realizamos la implementación de nuestra luz Spotlight la cual saldrá de nuestro cofre

```
spotLights[0] = Spotlight(1.0f, 0.6f, 1.0f, //color moradito
    1.0f, 2.0f,
    0.0f, 0.0f, 0.0f,
    -1.0f, 0.0f, 0.0f,
    1.0f, 0.0f, 0.0f,
    15.0f);
spotLightCount++;
```

Ahora, para que la dirección de nuestra luz cambie en ejecución al realizar a la vez el movimiento del cofre es necesario lo siguiente

```
// luz cofre
glm::vec3 linternaCofrePos = glm::vec3(-5.0f + mainWindow.getmovx(), 2.0f, 0.0f);
```

Aquí, especificamos el vector de dirección correctamente, algo parecido a lo que se había implementado en prácticas pasadas, sin embargo ahora limitamos la dirección de la luz a 90 grados.

```
glm::vec3 linternaCofreDir = glm::normalize(glm::vec3(
    -1.0f + abs(mainWindow.getarticulacionCofre() / 90.0f),
    abs(mainWindow.getarticulacionCofre() / 90.0f),
    0.0f));
```

También, en este bloque de código se hace la normalización al vector de posición, siendo que todo lo que se le suma al vector de posición Y, restarle el mismo valor al vector X.

Finalmente, renderizamos nuestro spotLight

```
(avanza) {
    spotLights1[0].SetFlash(linternaCofrePos, linternaCofreDir);
```

2. Agregar luz de tipo spotlight para el coche de tal forma que al avanzar (mover con teclado hacia dirección de X negativa) ilumine con un spotlight hacia adelante y al retroceder ((mover con teclado hacia dirección de X positiva) ilumine con un spotlight hacia atrás. Son dos spotlights diferentes que se prenderán y apagaran de acuerdo a alguna bandera asignada por ustedes.

Para este segundo ejercicio fue necesario crear dos arreglos lo cuales se les asignará una luz que corresponda a la dirección a la que nuestro modelo se mueva.

```
SpotLight spotLights1[MAX_SPOT_LIGHTS]; // luz delantera
SpotLight spotLights2[MAX_SPOT_LIGHTS]; // luz trasera
```

```
spotLights1[1] = SpotLight(0.0f, 0.3f, 1.0f,
    1.0f, 2.0f,
    0.0f, 0.0f, 0.0f,
    -1.0f, 0.0f, 0.0f,
    1.0f, 0.007f, 0.0002f,
    12.5f);
spotLightCount++;
```

```
spotLights1[2] = SpotLight(1.0f, 0.7f, 0.7f,
    1.0f, 2.0f,
    0.0f, 0.0f, 0.0f,
    1.0f, 0.0f, 0.0f,
    1.0f, 0.007f, 0.0002f,
    12.5f);
spotLightCount++;
```

Una vez que declaramos nuestros arreglos de luces, tenemos que agregar la lógica para nuestro modelo que al avanzar o retroceder se active, la luz trasera o delantera según corresponda, esto lo hacemos en Windows.h y Windows.cpp

```
GLfloat articulacionCofre;
int atr_adel;
bool mouseFirstMoved;
```

```
if (key >= 0 && key < 1024)
{
    if (action == GLFW_PRESS)
    {
        theWindow->keys[key] = true;
        theWindow->atr_adel = key;
    }
    else if (action == GLFW_RELEASE)
    {
        theWindow->keys[key] = false;
        theWindow->atr_adel = -1;
    }
}
```

Así pues, agregamos nuestras banderas para obtener información del movimiento y de el estado de nuestras luces

```
glm::mat4 projection = glm::perspective(40.0f,
    (float)theWindow->getWidth() / theWindow->getHeight(),
    0.1f, 100.0f);

bool bandera_mov = true;
bool bandera_luces[2] = { false, false };
// Lo que mientras no se cierre la ventana
```

Una vez implementado lo anterior ya podemos mandar a llamar a nuestra función para saber si el auto está retrocediendo o avanzando y así activar nuestras luces correspondientes

```
if (mainWindow.getTeclaPresionada() == GLFW_KEY_2) {
    shaderList[0].SetSpotLights(spotLights2, spotLightCount - 1);
    bandera_mov = false;
}
else {
    shaderList[0].SetSpotLights(spotLights1, spotLightCount - 1);
    bandera_mov = true;
}
```

```

if (bandera_mov) {
    spotLights1[0].SetFlash(linternaCofrePos, linternaCofreDir);
    // luz delantera
    spotLights1[1].SetFlash(glm::vec3(-5.0f + mainWindow.getmovx(), 1.5f, 0.0f)
        , glm::vec3(-1.0f, 0.0f, 0.0f));
}
else {
    spotLights2[0].SetFlash(linternaCofrePos, linternaCofreDir);
    // luz trasera
    spotLights2[1].SetFlash(glm::vec3(30.0f + mainWindow.getmovx(), 1.5f, 0.0f)
        , glm::vec3(1.0f, 0.0f, 0.0f));
}

```

3.- Agregar otra luz de tipo puntual ligada a un modelo elegido por ustedes (no lámpara) y que puedan prender y apagar de forma independiente con teclado tanto la luz de la lámpara como la luz de este modelo (la luz de la lámpara debe de ser puntual, si la crearon spotlight en su reporte 7 tienen que cambiarla a luz puntual)

Para este ultimo ejercicio, creamos 3 arreglos de luces tipo PointLight

```

PointLight pointLights1[MAX_POINT_LIGHTS];
PointLight pointLights2[MAX_POINT_LIGHTS];
PointLight pointLights3[MAX_POINT_LIGHTS];

```

Implementamos los arreglos

```

pointLights1[0] = PointLight(0.2f, 0.8f, 0.6f,
    0.0f, 1.0f,
    0.0f, 0.5f, 2.5f,
    0.3f, 0.2f, 0.1f);
pointLightCount++;

pointLights2[0] = PointLight(0.8f, 0.95f, 0.75f,
    0.0f, 1.0f,
    10.0f, -0.1f, -4.0,
    1.0f, 0.09f, 0.032f);
pointLightCount++;

pointLights3[0] = pointLights1[0];
pointLights3[1] = pointLights2[0];

```

Siguiendo con la misma lógica del ejercicio anterior, reusamos nuestra bandera para activar/desactivar nuestras luces

```

bool bandera_luces[2] = { false, false };

```

Siguiendo con el mismo esquema de ejercicios anteriores, declaramos la lógica de el encendido y apagado de nuestras luces.

```

if (mainWindow.getTeclaPresionada() == GLFW_KEY_3) {
    bandera_luces[0] = true;
}
if (mainWindow.getTeclaPresionada() == GLFW_KEY_4) {
    bandera_luces[0] = false;
}

if (mainWindow.getTeclaPresionada() == GLFW_KEY_5) {
    bandera_luces[1] = true;
}
if (mainWindow.getTeclaPresionada() == GLFW_KEY_6) {
    bandera_luces[1] = false;
}

```

Y por último, mandamos la información al shader

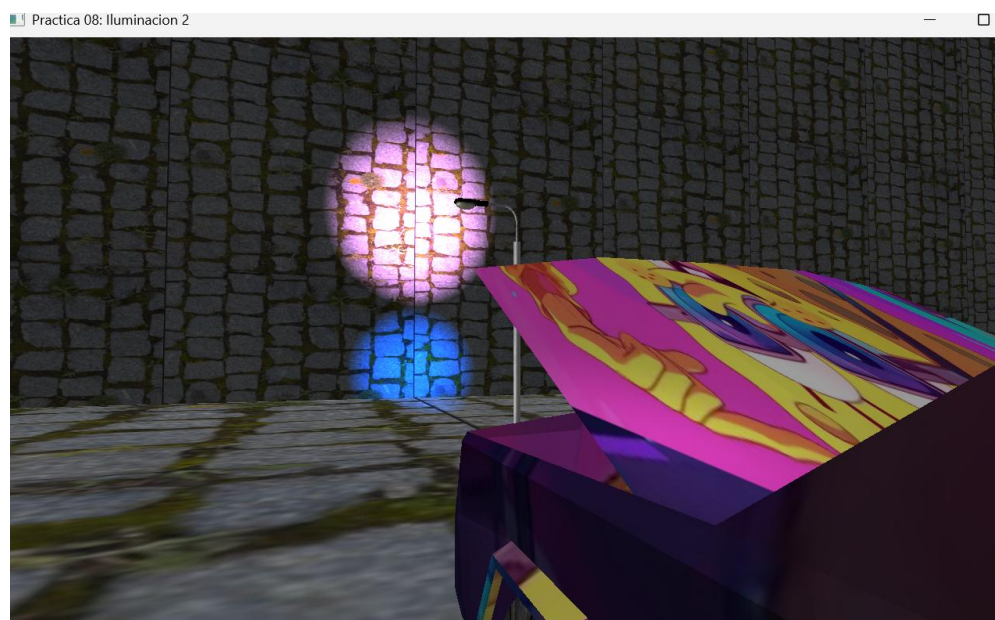
```

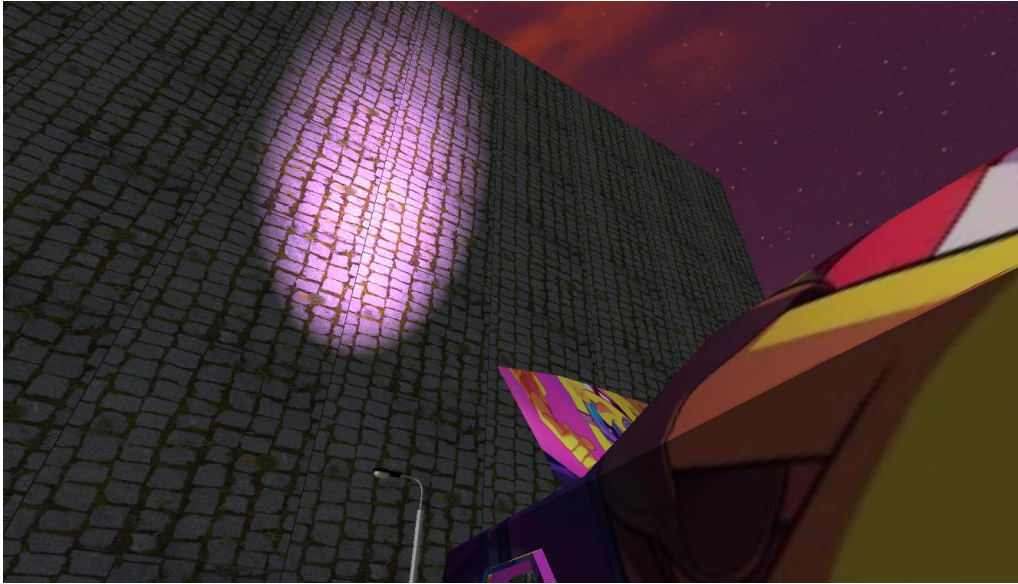
if (bandera_luces[0] && !bandera_luces[1]) {
    shaderList[0].SetPointLights(pointLights1, 1);
}
if (bandera_luces[1] && !bandera_luces[0]) {
    shaderList[0].SetPointLights(pointLights2, 1);
}
if (bandera_luces[0] && bandera_luces[1]) {
    shaderList[0].SetPointLights(pointLights3, pointLightCount);
}
if (!bandera_luces[0] && !bandera_luces[1]) {
    shaderList[0].SetPointLights(pointLights3, 0);
}

```

CAPTURAS DE EJECUCIÓN:

EJERCICIO 1.-



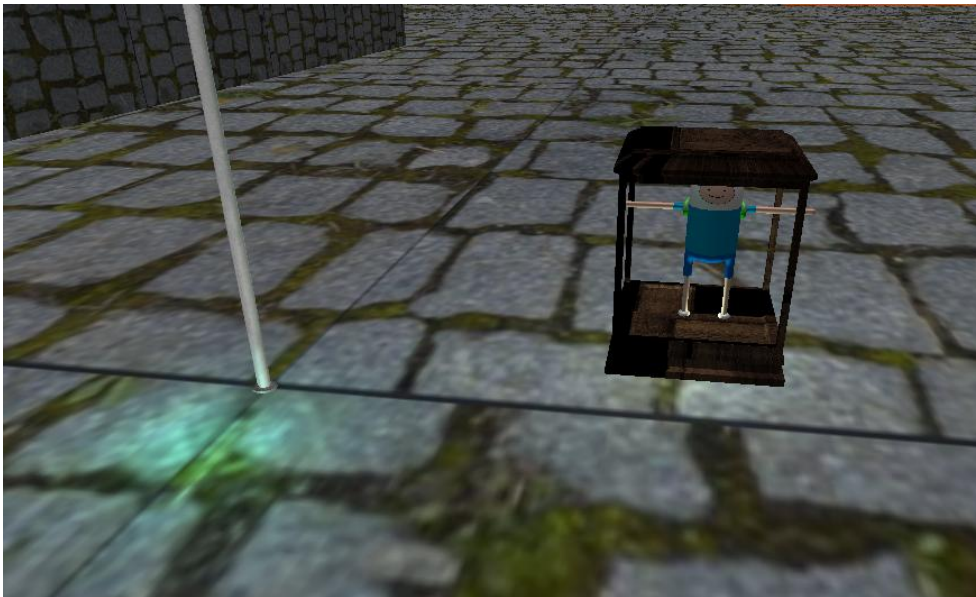


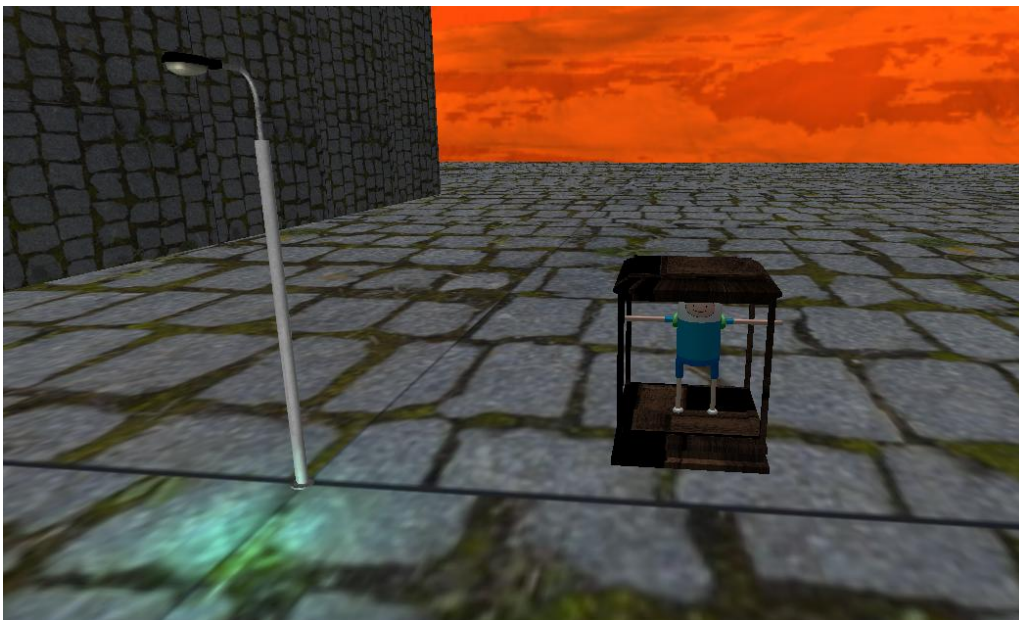
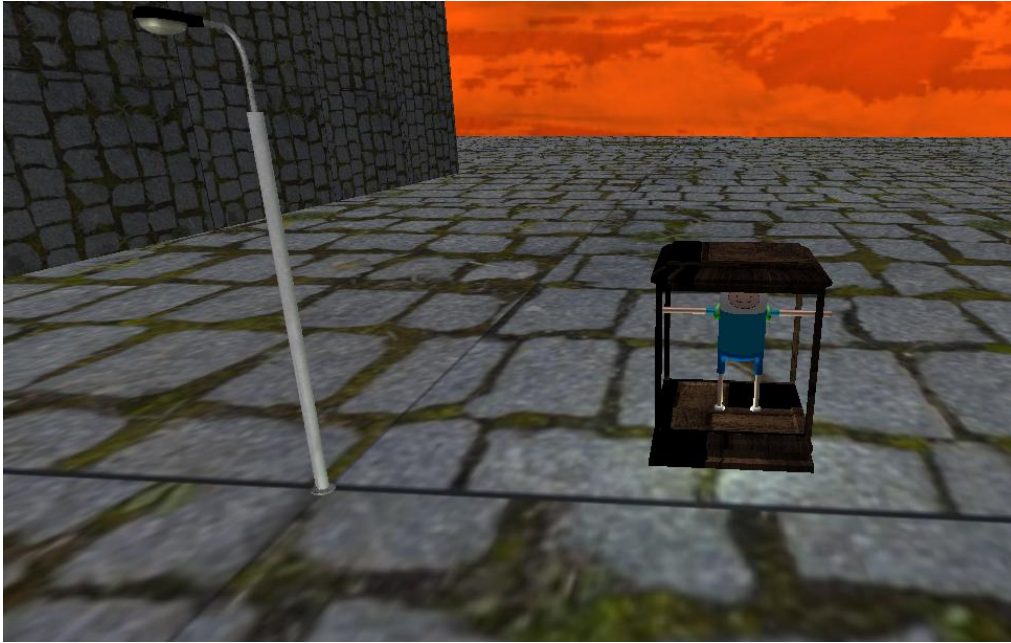
EJERCICIO 2.-





EJERCICIO 3.-





PROBLEMAS SURGIDOS:

Afortunadamente, en esta práctica no se tuvo ningún problema.

CONCLUSIÓN

En esta practica se mostró la importancia de llevar un hilo de los ejercicios, ejemplos e implementaciones anteriores, ya que se reusó gran parte de la lógica que ya se venía implementando anteriormente, sin embargo, el control de

SpotLighst, por ejemplo, para que éstas cambiaran depende de una entrada de teclado fue un reto, ya que se necesitaba de nuevas implementaciones.

Todo lo anterior fue de gran importancia para comprender varios flujos de código en OpenGL y poner en práctica la base para futuros proyectos en relación con este tema.